

# Identification de systèmes d'état non linéaires basée sur l'apprentissage profond

Mohammed El Hadi Zerga

24 septembre 2024

Nous examinons un système non linéaire à temps discret caractérisé par:

$$\begin{cases} x_{t+1} = f^o(x_t, u_t) \\ y_t = h^o(x_t, u_t) + w_t, \end{cases} \quad (1)$$

- Variables:

- État  $x_t$  dans  $\mathcal{X} \subset \mathbb{R}^{n_x}$
- Entrée  $u_t$  dans  $\mathcal{U} \subset \mathbb{R}^{n_u}$
- Sortie  $y_t$  dans  $\mathcal{Y} \subset \mathbb{R}^{n_y}$

- Fonctions non linéaires:

- $f^o: \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \rightarrow \mathbb{R}^{n_x}$
- $h^o: \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \rightarrow \mathbb{R}^{n_y}$

- Bruit de mesure  $w_t$  dans  $\mathcal{W} \subset \mathbb{R}^{n_y}$

- ① **Ensembles compacts** : Les signaux  $u$  et  $w$  sont dans les ensembles compacts  $\mathcal{U}$  et  $\mathcal{W}$  respectivement.
- ② **Espace d'états** :  $\mathcal{X}$  est un ensemble compact contenant l'état initial  $x_0$ .
- ③ **Conditions d'invariance** :
  - $f^o(x, u)$  reste dans  $\mathcal{X}$  pour tout  $(x, u)$  dans  $\mathcal{X} \times \mathcal{U}$ .
  - $h^o(x, u) + w$  appartient à  $\mathcal{Y}$  pour tout  $(x, u, w)$  dans  $\mathcal{X} \times \mathcal{U} \times \mathcal{W}$ .
- ④ **Lipschitzienne** :  $f^o$  et  $h^o$  sont uniformément Lipschitzienne sur  $\mathcal{X} \times \mathcal{U}$ , avec une constante  $\gamma_f$  telle que :

$$\|f^o(x, u) - f^o(x', u)\| \leq \gamma_f \|x - x'\|$$

pour tout  $(x, x', u)$ .

Supposons qu'un problème puisse être modélisé par le système (1) et que nous disposions de  $N$  paires de données d'entrée et de sortie

$$\{(u_t, y_t) : t = 1, \dots, N\}.$$

Notre objectif est d'estimer le système (1) associé à notre problème.

- ❶ **Modélisation** : Construction d'un modèle de régression fonctionnelle non linéaire basé sur les équations du système.
- ❷ **Apprentissage** : Développement et entraînement d'une structure de réseau neuronal pour estimer la fonction non linéaire associée au problème de régression.
- ❸ **Réduction de modèle** : Utilisation d'un encodeur-décodeur pour obtenir une représentation d'état plus compacte, malgré une dimension potentiellement élevée.

# Expression de l'État en Fonction des Entrées et Sorties Passées

Nous allons d'abord établir que, sous certaines conditions, nous pouvons écrire la variable d'état en fonction des entrées et des sorties, telle que

$$x_t = \phi(z_t) \quad (2)$$

où

$$z_t = (u_{t-\ell}^\top \quad y_{t-\ell}^\top \quad \cdots \quad u_{t-1}^\top \quad y_{t-1}^\top)^\top \quad (3)$$

$\phi$  est une fonction non linéaire et  $\ell \in \mathbb{N}$

## Définition 1: Observabilité Finie

Le système (1) est dit finiment observable sur un horizon temporel  $i \in \mathbb{N}$  si pour chaque  $\bar{u} \in \mathcal{U}^r$ , la fonction

$$O_i(x, \bar{u}_{1|i}) = \begin{pmatrix} h^o(x, u_1) \\ h^o(F_1(x, u_1), u_2) \\ \vdots \\ h^o(F_{i-1}(x, u_1, \dots, u_{i-1}), u_i) \end{pmatrix}. \quad (6)$$

est injective.

ou pour tout  $i \geq 1, F_i(x, \bar{u}_{1|i}) = f^o(F_{i-1}(x, \bar{u}_{1|i-1}), u_i)$  avec

$$\bar{u}_{i|j} = (u_i^\top \quad \cdots \quad u_j^\top)^\top$$

### Proposition 1 : Existence de la fonction $\phi$

Si le système non linéaire (1) (considéré sous l'hypothèse que  $w \equiv 0$ ) est finiment observable au sens de la Définition 1, alors il existe  $\ell \in \mathbb{N}$  et une fonction (non linéaire)  $\phi : \mathbb{R}^L \rightarrow \mathbb{R}^{n_x}$  telle que  $x_t = \phi(z_t)$  pour tout temps  $t \geq \ell$ , tout état initial  $x \in \mathcal{X}$  et tout signal d'entrée prenant des valeurs dans  $\mathcal{U}$ .

De plus si  $w \neq 0$  on peut définir  $\hat{x}_{t-\ell}$  et  $\hat{x}_t$  comme ceci :

$$\hat{x}_{t-\ell} \in \arg \min_{x \in \mathcal{X}} \left\| \bar{y}_{t-\ell|t-1} - \mathcal{O}_\ell(x, \bar{u}_{t-\ell|t-1}) \right\| \quad (7)$$

$$\hat{x}_t = F_\ell(\hat{x}_{t-\ell}, \bar{u}_{t-\ell|t-1}) \quad (8)$$

## Définition 2

Le système (1) est dit finiment uniformément observable sur un horizon temporel  $\ell \in \mathbb{N}$  s'il existe une constante  $\alpha_\ell > 0$  telle que pour chaque  $\bar{u} \in \mathcal{U}^\ell$ ,

$$\|O_\ell(x, \bar{u}) - O_\ell(x', \bar{u})\| \geq \alpha_\ell \|x - x'\| \quad \text{pour tous } (x, x') \in \mathbb{R}^{n_x} \times \mathbb{R}^{n_x}. \quad (9)$$

## Proposition 2

Sous les hypothèses 1 à 4, si le système (1) est finiment uniformément observable sur un horizon temporel  $\ell \in \mathbb{N}$  au sens de la Définition 2, alors

$$\|\hat{x}_t - x_t\| \leq 2\gamma_f^\ell \alpha_\ell^{-1} \|\bar{w}_{t-\ell|t-1}\| \quad (10)$$

où  $\gamma_f$  est la constante de Lipschitz de  $f^o$  et  $\alpha_\ell$  est la constante apparaissant dans (9).



Posons  $x_t = \hat{x}_t + \delta_t$  . on remplace dans (1)

$$\begin{aligned} y_t &= h^o(\hat{x}_t + \delta_t, u_t) + w_t \\ &= h^o(\hat{x}_t, u_t) + \xi_t, \end{aligned}$$

avec  $\xi_t$  étant une composante d'erreur entièrement due au bruit, de plus puisque  $\hat{x}_t$  est une fonction de  $z_t$ , nous obtenons

$$y_t = H^o(z_t, u_t) + \xi_t \quad (11)$$

pour une certaine fonction non linéaire  $H^o$ .

Considérons la représentation d'état :

$$\begin{cases} z_{t+1} = \bar{A}z_t + \bar{B}u_t + \bar{S}H^o(z_t, u_t) + \bar{S}\xi_t \\ y_t = H^o(z_t, u_t) + \xi_t, \end{cases} \quad (12)$$

L'estimation de  $H^o$  est formulée comme un problème d'optimisation :

$$\hat{H} \in \arg \min_{H \in \mathcal{H}} J(H),$$

où la fonction de perte  $J(H)$  est définie par :

$$J(H) = \frac{1}{T} \sum_{t=\ell+1}^{T+\ell} \alpha_t \|y_t - H(\hat{z}_t, u_t)\|^2,$$

et l'évolution de l'état est donnée par :

$$\hat{z}_{t+1} = \bar{A}\hat{z}_t + \bar{B}u_t + \bar{S}H(\hat{z}_t, u_t), \quad \hat{z}_{\ell+1} = z_{\ell+1}.$$

construire l'auto-encodeur  $(\mathcal{E}, \mathcal{D})$  tel que

$$\bar{x}_t = \mathcal{E}(z_t)$$

$$\bar{z}_t = \mathcal{D}(\bar{x}_t).$$

En appliquant ces transformations à l'équation (12) :

$$\begin{cases} \bar{x}_{t+1} = \mathcal{E} \left( \bar{A}\mathcal{D}(\bar{x}_t) + \bar{B}u_t + \bar{S}\hat{H}(\mathcal{D}(\bar{x}_t), u_t) \right) \\ \bar{y}_t = \hat{H}(\mathcal{D}(\bar{x}_t), u_t). \end{cases} \quad (15)$$

$$(\mathcal{E}, \mathcal{D}) = \arg \min_{\mathcal{E}', \mathcal{D}'} \|z_t - \mathcal{D}'(\mathcal{E}'(z_t))\|. \quad (16)$$

nous sélectionnons plusieurs conditions initiales

$$\{x_{0,i} : i = 1, \dots, d\}$$

Nous disposons ainsi pour chaque condition initial  $x_{0,i}$  les données suivantes :

$$D_i := \{(u_t, y_t) : t = 1, \dots, k\}$$

avec  $k \in \mathbb{N}^*$ . Nous diviserons les  $D_i$  en données de test et en données d'entraînement comme suit :

$$E = \{D_i : i = 1, \dots, m\}$$

pour entraîner notre modèle et

$$T = \{D_i : i = m + 1, \dots, d\}$$

pour le tester.

$$\hat{H} \in \arg \min_{H \in \mathcal{H}} J(H),$$

L'espace  $\mathcal{H}$  est défini par l'ensemble de toutes les fonctions  $f$  réalisables par les configurations possibles des poids et biais du réseau :

$$\mathcal{H} = \left\{ f \mid f(x) = W^{n+1} \sigma \left( W^n \sigma \left( \dots \sigma \left( W^1 x + b^1 \right) \dots \right) + b^n \right) + b^{n+1}, W^l \in \mathbb{R}^{m \times m}, b^l \in \mathbb{R}^m \right\}$$

# Construction de $J$

$$J(H) = \frac{1}{T} \sum_{t=\ell+1}^{T+\ell} \alpha_t \|y_t - H(\hat{z}_t, u_t)\|^2,$$

$$\hat{z}_{t+1} = \bar{A}\hat{z}_t + \bar{B}u_t + \bar{S}H(\hat{z}_t, u_t), \quad \hat{z}_{\ell+1} = z_{\ell+1}.$$

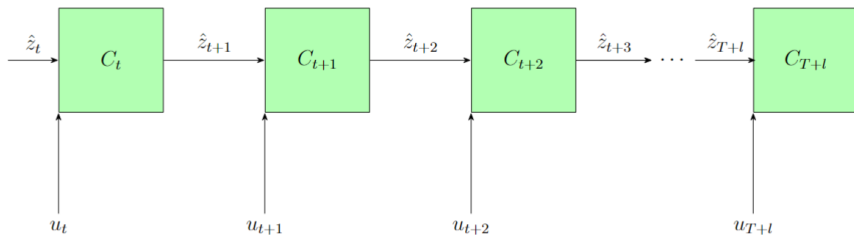


Figure 4: Procédure de mise à jour de  $\hat{z}_t$

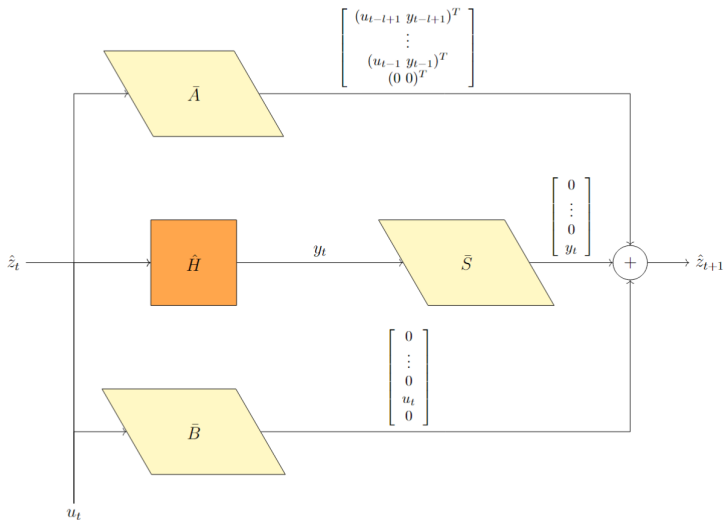


Figure 5: Opération effectuée dans le bloc  $C_t$

# Minimisation de $J$

Supposons que nous ayons initialisé l'ensemble des paramètres de notre réseau de neurones, et notons-les  $\theta_0$ . Nous avons donc :

$$J(\theta_0) = \frac{1}{T} \sum_{t=\ell+1}^{T+\ell} \alpha_t \|y_t - H(\hat{z}_t, u_t, \theta_0)\|^2.$$

en utilisant un algorithme basé sur la descente de gradient, la fonction  $J$  est mise à jour comme suit :

$$J(\theta_1) = \frac{1}{T} \sum_{t=\ell+1}^{T+\ell} \alpha_t \|y_t - H(\hat{z}_t, u_t, \theta_1)\|^2,$$

puis

$$J(\theta_2) = \frac{1}{T} \sum_{t=\ell+1}^{T+\ell} \alpha_t \|y_t - H(\hat{z}_t, u_t, \theta_2)\|^2,$$

et ainsi de suite, jusqu'à ce que la valeur de la fonction  $J$  soit suffisamment faible selon nos critères.



$$\bar{x}_t = \mathcal{E}(z_t)$$
$$\bar{z}_t = \mathcal{D}(\bar{x}_t).$$

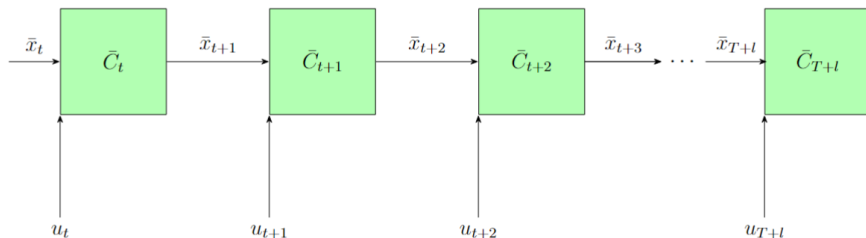


Figure 6: Procédure de mise à jour de  $\hat{z}_t$

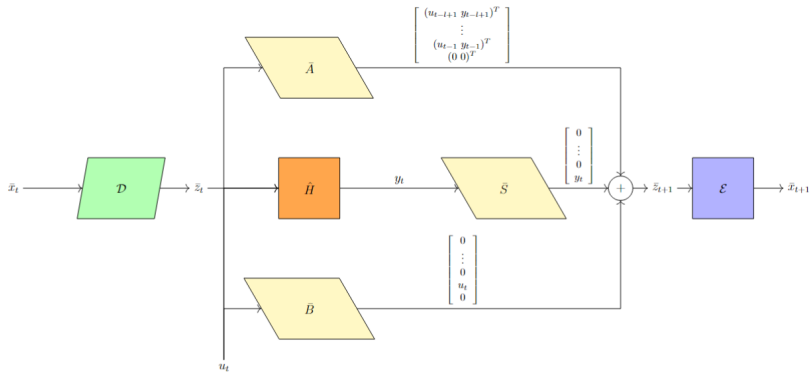


Figure 7: Opération effectuée dans le bloc  $C_t$

## **Jax**

Une bibliothèque de Google combinant NumPy et le compilateur XLA, offrant différenciation automatique et calcul parallèle (GPU/TPU) pour l'apprentissage automatique.

## **Flax**

Basée sur JAX, facilite le développement de modèles de réseaux neuronaux avec des abstractions simples pour les couches et les optimisateurs, similaire à PyTorch.

## **Scan**

La fonction scan de JAX permet d'effectuer des opérations itératives de manière efficace et vectorisée. Elle est utile pour des calculs où chaque sortie dépend de la précédente, comme les sommes ou produits cumulés.

$$x[k + 1] = Ax[k] + Bu[k]$$

$$y[k] = Cx[k] + Du[k]$$

$$A = \begin{bmatrix} 1.13 & 0.9774 & -1.027 & -0.7225 & 0.6295 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} \quad B = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

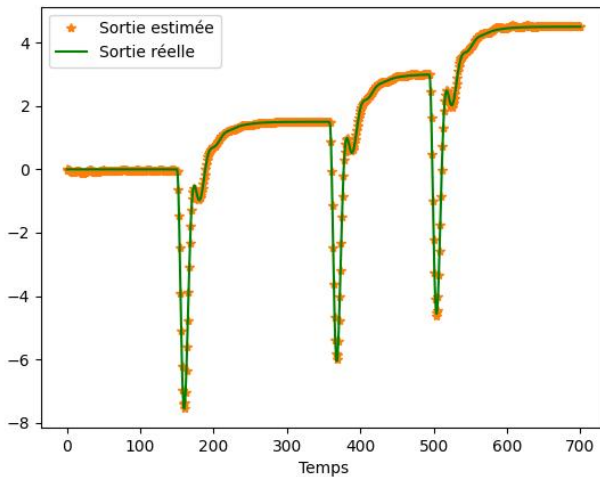
$$C = [-0.5108 \quad -0.3642 \quad 0.3828 \quad 0.4666 \quad 0.04719]$$

$$D = [-0.124]$$

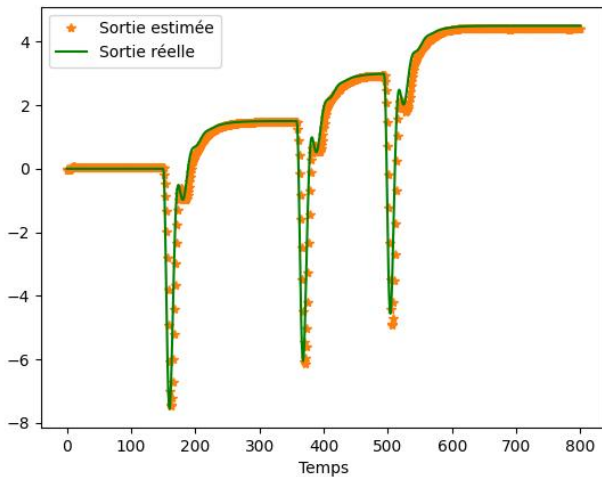
Afin d'entraîner notre modèle, nous allons générer des données en prenant  $u$  tel que :

$$u[k] = \begin{cases} 0 & \text{si } k < t_1 \\ 1 & \text{si } t_1 \leq k < t_2 \\ 2 & \text{si } t_2 \leq k < t_3 \\ 3 & \text{si } k \geq t_3 \end{cases}$$

# Résultat sans auto encoder



# Résultat avec auto encoder

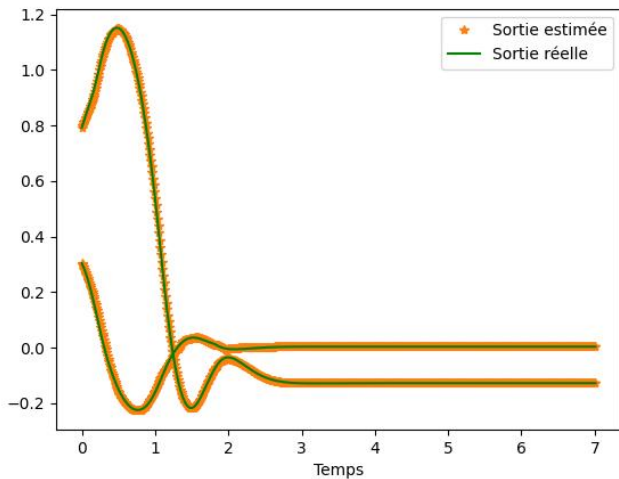


$$\dot{\xi} = A\xi + B_1\delta + B_2\dot{\psi}_{des} + B_3\ddot{\psi}_{des} + Ff \quad (4)$$

$$z = C\xi \quad (5)$$



# Résultat sans auto encoder



# Résultat avec auto encoder

